

SimKaal

Présentation des TP

Alain Bannay

École Nationale de l'Aviation Civile
Sub SINA/INF/SAR

Les thèmes

- 1 Prise en main et modes d'adressage
- 2 Sous-programme et pile
- 3 Unité d'échange et transfert par mot d'état
- 4 Les interruptions
- 5 Synthèse

1 Prise en main et modes d'adressage

- Rappels sur Simkaal
- Modes d'adressage de SimKaal
- Les branchements
- Les étapes de programmation

2 Sous-programme et pile

3 Unité d'échange et transfert par mot d'état

4 Les interruptions

5 Synthèse

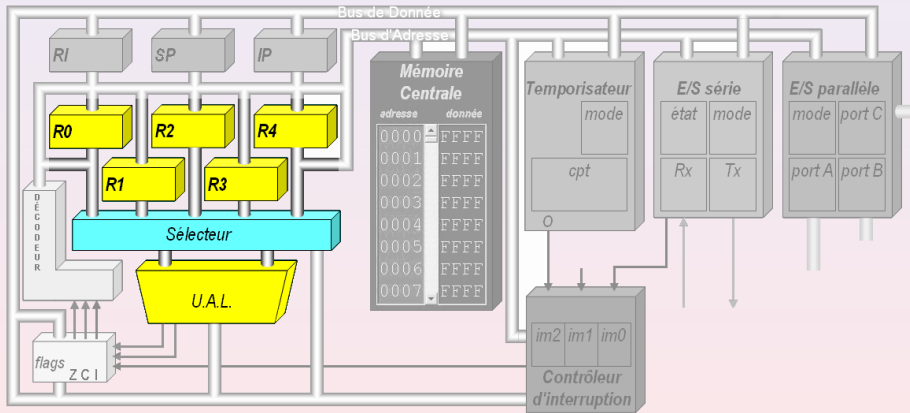


Figure – Le bloc de calcul : UAL, Registres et Sélecteur

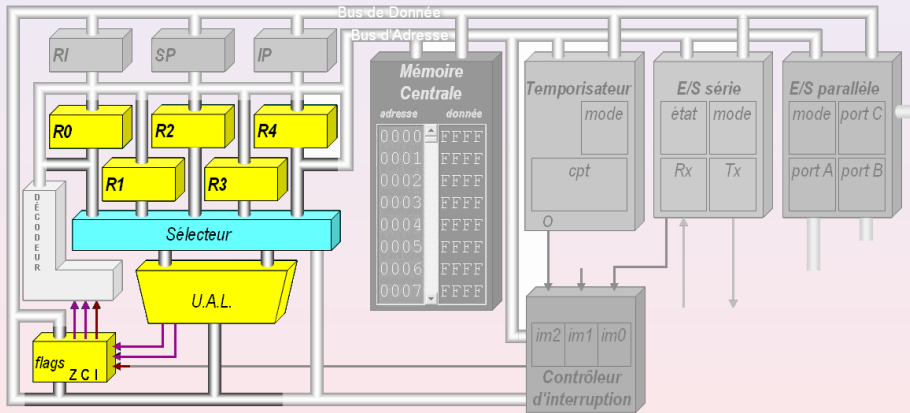


Figure – Le bloc de calcul : Flags

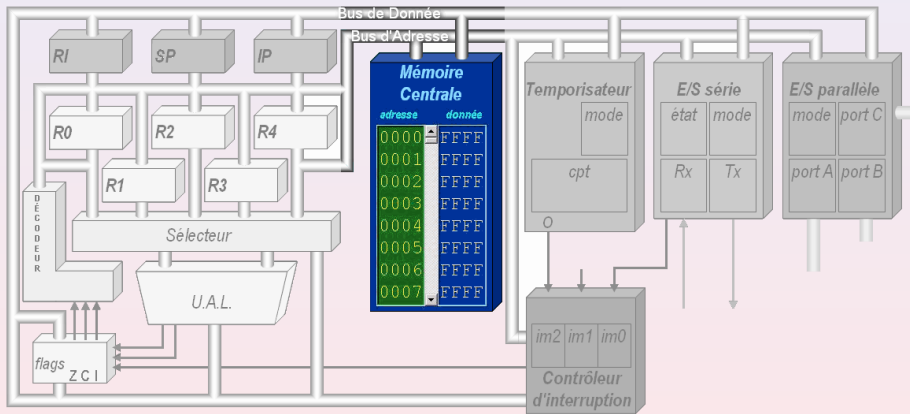


Figure – La mémoire centrale

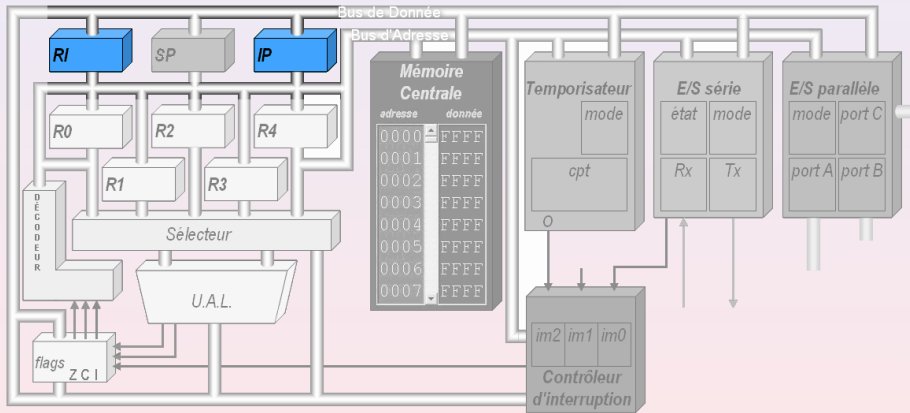


Figure – Le bloc de commande : RI, IP

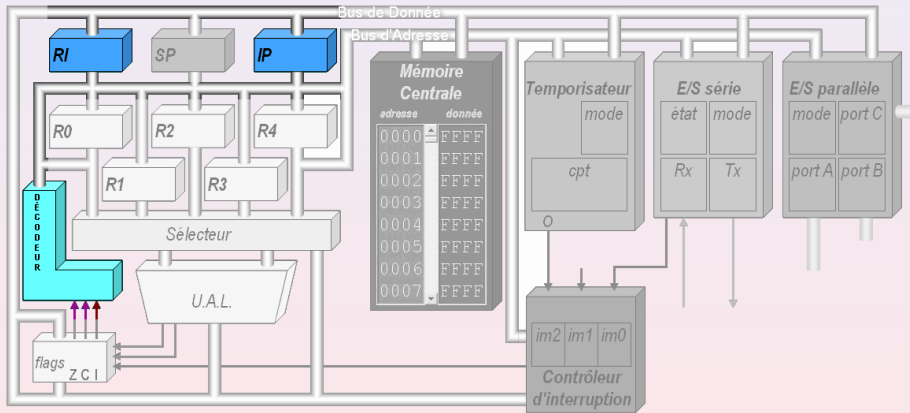


Figure – Le bloc de commande : Décodeur, Séquenceur

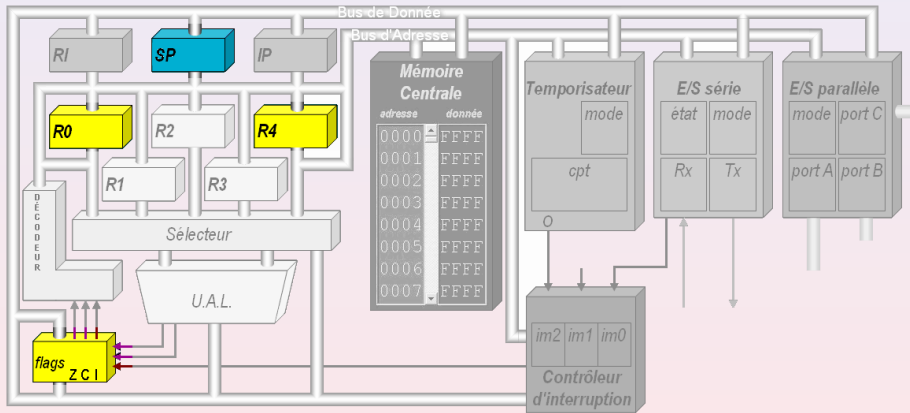


Figure – Les registres spécialisés : SP, R0, R4, FL

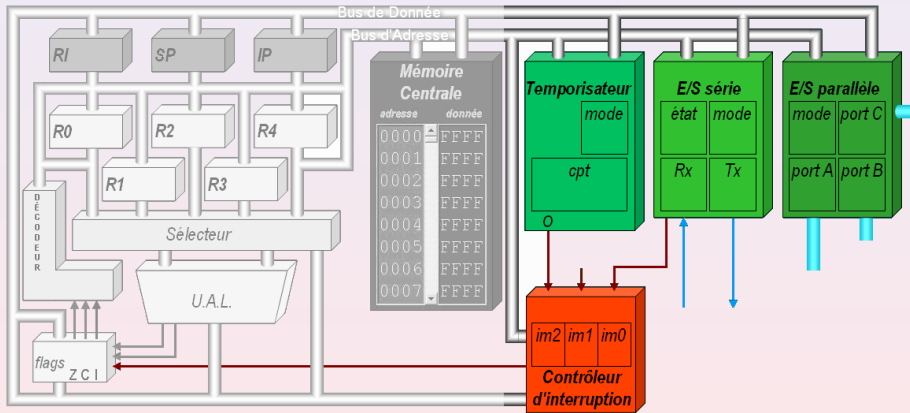


Figure – L'unité d'échange

Généralités

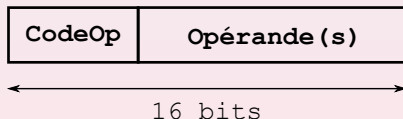
- Dans une instruction, on doit trouver :
 - ▶ le type de l'opération à effectuer (CodeOp),
 - ▶ éventuellement une donnée nécessaire à l'exécution de l'opération (Operande(s)).

Généralités

- Dans une instruction, on doit trouver :
 - ▶ le type de l'opération à effectuer (CodeOp),
 - ▶ éventuellement une donnée nécessaire à l'exécution de l'opération (Operande(s)).
- SimKaal s'appuie sur une architecture RISC
- Toutes les instructions sont codées sur 16 bits.

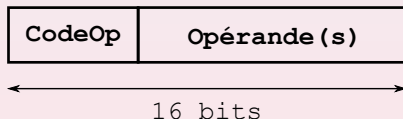
Généralités

- Dans une instruction, on doit trouver :
 - ▶ le type de l'opération à effectuer (CodeOp),
 - ▶ éventuellement une donnée nécessaire à l'exécution de l'opération (Opérande(s)).
- SimKaal s'appuie sur une architecture RISC
- Toutes les instructions sont codées sur 16 bits.



Généralités

- Dans une instruction, on doit trouver :
 - ▶ le type de l'opération à effectuer (CodeOp),
 - ▶ éventuellement une donnée nécessaire à l'exécution de l'opération (Opérande(s)).
- SimKaal s'appuie sur une architecture RISC
- Toutes les instructions sont codées sur 16 bits.

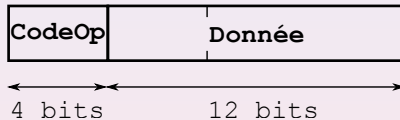


Mode d'adressage

Méthode permettant d'accéder aux opérandes.

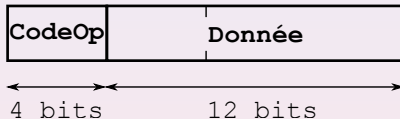
Adressage immédiat

- L'instruction contient l'opérande sous forme d'une constante.
- Format :



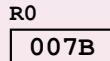
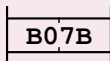
Adressage immédiat

- L'instruction contient l'opérande sous forme d'une constante.
- Format :



- Exemples :

```
ldr r0,123
```

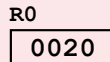
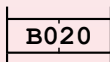


```
ORG 0x20
```

```
res: CST 3
```

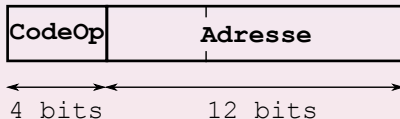
```
...
```

```
ldr r0,res
```



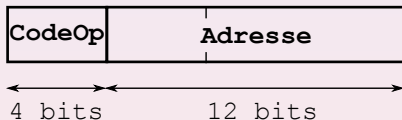
Adressage direct

- L'instruction contient *l'adresse* de l'opérande.
- Accès à l'opérande $\Rightarrow$ 1 accès mémoire
- En assembleur : [...] $\Rightarrow$ 'contenu de ...'
- Format :



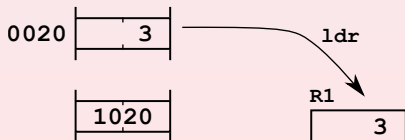
Adressage direct

- L'instruction contient *l'adresse* de l'opérande.
- Accès à l'opérande $\Rightarrow$ 1 accès mémoire
- En assembleur : [...] $\Rightarrow$ 'contenu de ...'
- Format :



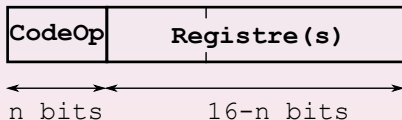
- Exemple :

```
ORG 0x20
res: CST 3
...
ldr r1, [res]
```



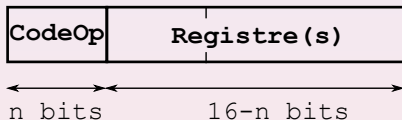
Adressage par registre

- L'opérande se trouve dans un registre.
- L'instruction contient une référence à ce registre.
- Accès à un opérande $\Rightarrow$ 1 accès registre.
- Format :



Adressage par registre

- L'opérande se trouve dans un registre.
- L'instruction contient une référence à ce registre.
- Accès à un opérande $\Rightarrow$ 1 accès registre.
- Format :



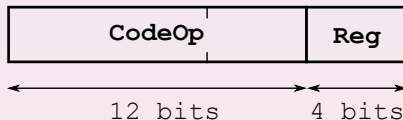
- Exemple :

```
inc r0  
ldr r1,r4  
add r2,r3r4
```

FE10
D014
E234

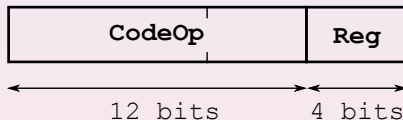
Adressage indirect par registre

- L'*adresse* de l'opérande se trouve dans un registre.
- L'instruction contient une référence à ce registre.
- Accès à l'opérande $\Rightarrow$ 1 accès registre + 1 accès mémoire.
- Format :



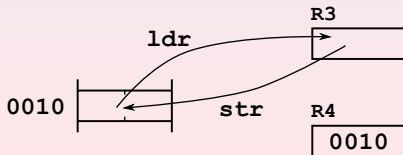
Adressage indirect par registre

- L'*adresse* de l'opérande se trouve dans un registre.
- L'instruction contient une référence à ce registre.
- Accès à l'opérande $\Rightarrow$ 1 accès registre + 1 accès mémoire.
- Format :



- Exemple :

```
ldr r3, [r4]
str [r4], r3
```



Adressage relatif

- Utilisé dans les instructions de branchement.
- L'instruction contient un offset à ajouter ou retirer à l'adresse courante pour obtenir l'adresse de branchement.
- En assembleur : `jmp Etiquette`
- Format :



Adressage relatif

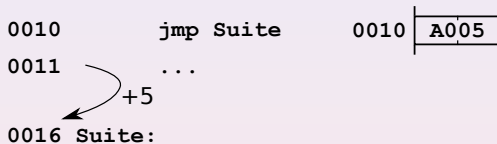
- Utilisé dans les instructions de branchement.
- L'instruction contient un offset à ajouter ou retirer à l'adresse courante pour obtenir l'adresse de branchement.
- En assembleur : `jmp Etiquette`
- Format :



- Exemples :
 `jmp debut`
 `call sousprog`
 `jc carry`

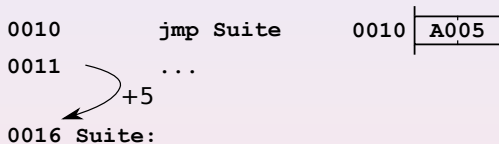
Adressage relatif

- Branchement inconditionnel avec offset positif :

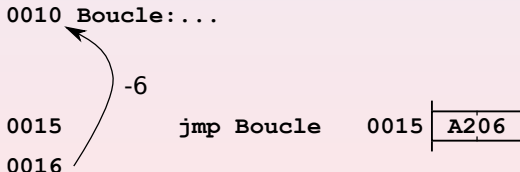


Adressage relatif

- Branchement inconditionnel avec offset positif :



- Branchement inconditionnel avec offset négatif :



Instructions de branchement sous SimKaal

Branchement inconditionnel :

- Saut : `jmp`
- Appel de sous-programme : `call`

Instructions de branchement sous SimKaal

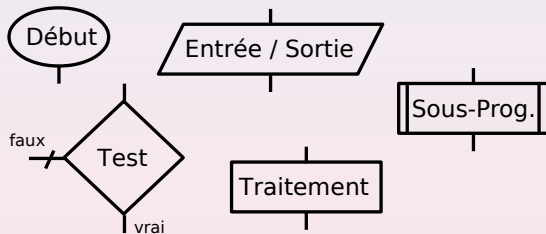
Branchement inconditionnel :

- Saut : `jmp`
- Appel de sous-programme : `call`

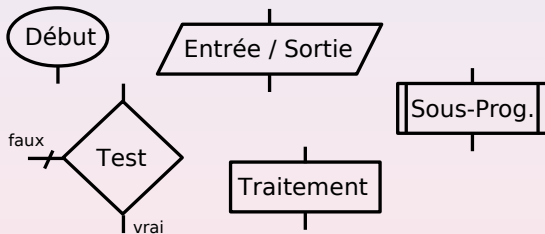
Branchement conditionnel :

- Selon l'état de la carry : `jc`, `jnc`
- Par rapport à la valeur zéro : `jz`, `jnz`
- **C** et **Z** mis à jour par l'ALU.

- 1 Mise en organigramme de ce qui est demandé.

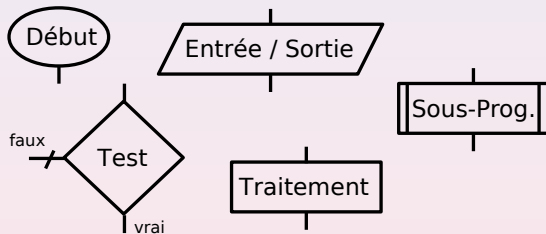


- 1 Mise en organigramme de ce qui est demandé.



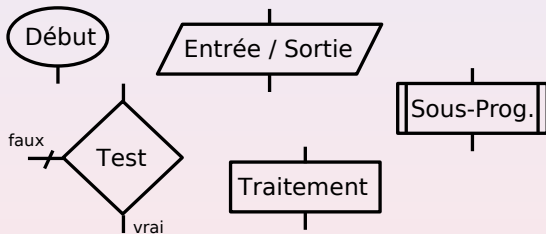
- 2 Écriture en langage SimKaa1 (fichier .asm).

- 1 Mise en organigramme de ce qui est demandé.



- 2 Écriture en langage SimKaal (fichier .asm).
- 3 Génération du code machine via AsmKaal (fichiers .sim et .lst).

- 1 Mise en organigramme de ce qui est demandé.



- 2 Écriture en langage SimKaal (fichier .asm).
- 3 Génération du code machine via AsmKaal (fichiers .sim et .lst).
- 4 Charger le .sim dans SimKaal et l'exécuter.

- 1 Prise en main et modes d'adressage
- 2 **Sous-programme et pile**
 - Sous-programme
 - Pile
 - Utilisation
- 3 Unité d'échange et transfert par mot d'état
- 4 Les interruptions
- 5 Synthèse

Principe

Exécuter un bloc d'instructions sur ordre d'un autre programme

Principe

Exécuter un bloc d'instructions sur ordre d'un autre programme

Programme



Fin

Principe

Exécuter un bloc d'instructions sur ordre d'un autre programme

Programme



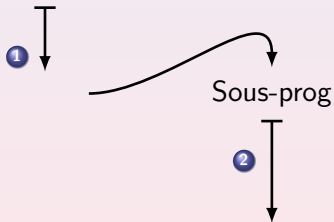
- 1 Stopper l'exécution séquentielle du programme

Fin

Principe

Exécuter un bloc d'instructions sur ordre d'un autre programme

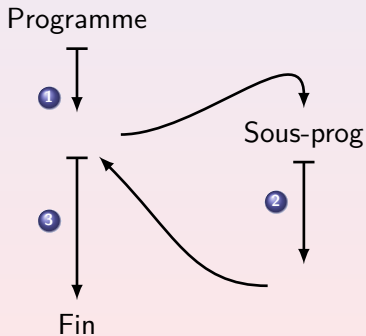
Programme



- 1 Stopper l'exécution séquentielle du programme
- 2 Exécuter le sous-programme

Principe

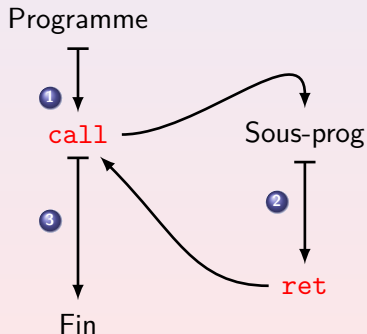
Exécuter un bloc d'instructions sur ordre d'un autre programme



- 1 Stopper l'exécution séquentielle du programme
- 2 Exécuter le sous-programme
- 3 Reprendre l'exécution du programme appelant

Principe

Exécuter un bloc d'instructions sur ordre d'un autre programme



- 1 Stopper l'exécution séquentielle du programme
- 2 Exécuter le sous-programme
- 3 Reprendre l'exécution du programme appelant

Instructions relatives aux procédures

Instructions relatives aux procédures

`call SousProg`

- Sauvegarde en *pile* de l'adresse de retour
- Branchement à la 1^{ère} instruction de SousProg

Instructions relatives aux procédures

`call SousProg`

- Sauvegarde en *pile* de l'adresse de retour
- Branchement à la 1^{ère} instruction de SousProg

`ret`

- Adressage implicite (pas d'opérande)
- Restitution de l'adresse de retour (dans le registre IP)

Définition de la pile

- Zone mémoire destinée à l'empilement d'informations

Définition de la pile

- Zone mémoire destinée à l'empilement d'informations
- Registre associé : Stack Pointer (SP)

Définition de la pile

- Zone mémoire destinée à l'empilement d'informations
- Registre associé : Stack Pointer (SP)
- Gestion côté programmeur :
 - ▶ choix de l'emplacement et de la taille
 - ▶ initialisation de SP :
`ldr r0,0x100`
`ldr sp,r0`
 - ▶ utilisation : **push** et **pop**

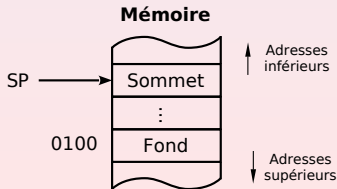
Définition de la pile

- Zone mémoire destinée à l'empilement d'informations
- Registre associé : Stack Pointer (SP)
- Gestion côté programmeur :
 - ▶ choix de l'emplacement et de la taille
 - ▶ initialisation de SP :
`ldr r0,0x100`
`ldr sp,r0`
 - ▶ utilisation : **push** et **pop**
- Gestion côté unité centrale :
 - ▶ mise à jour de SP

Définition de la pile

- Zone mémoire destinée à l'empilement d'informations
- Registre associé : Stack Pointer (SP)
- Gestion côté programmeur :
 - ▶ choix de l'emplacement et de la taille
 - ▶ initialisation de SP :

```
ldr r0,0x100
ldr sp,r0
```
 - ▶ utilisation : **push** et **pop**
- Gestion côté unité centrale :
 - ▶ mise à jour de SP
- Représentation :

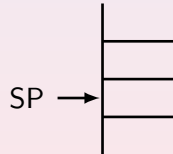


Instructions relatives à la pile

push r0

Registre R0

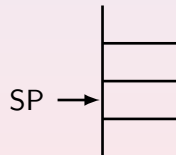
Val.



Instructions relatives à la pile

`push r0`

① $SP = SP - 1$



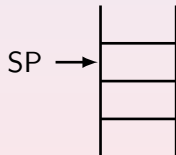
Registre R0

Val.

Instructions relatives à la pile

`push r0`

① $SP = SP - 1$



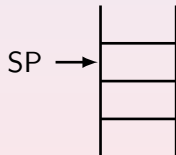
Registre R0

Val.

Instructions relatives à la pile

`push r0`

- 1 $SP = SP - 1$
- 2 Empiler valeur



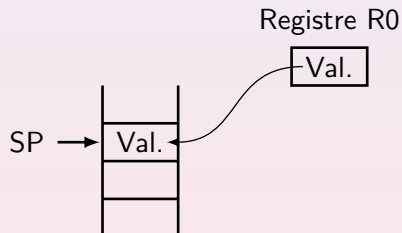
Registre R0

Val.

Instructions relatives à la pile

`push r0`

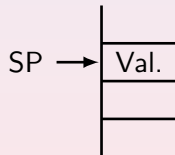
- 1 $SP = SP - 1$
- 2 Empiler valeur



Instructions relatives à la pile

`push r0`

- 1 $SP = SP - 1$
- 2 Empiler valeur



Registre R0

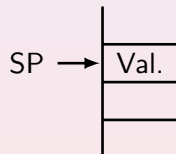
Val.

Instructions relatives à la pile

`push r0`

- 1 $SP = SP - 1$
- 2 Empiler valeur

`pop r0`



Registre R0



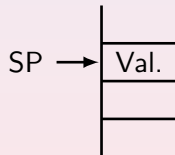
Instructions relatives à la pile

`push r0`

- 1 $SP = SP - 1$
- 2 Empiler valeur

`pop r0`

- 1 Dépiler valeur



Registre R0



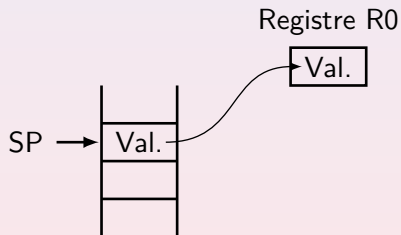
Instructions relatives à la pile

`push r0`

- 1 $SP = SP - 1$
- 2 Empiler valeur

`pop r0`

- 1 Dépiler valeur



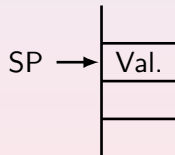
Instructions relatives à la pile

`push r0`

- 1 $SP = SP - 1$
- 2 Empiler valeur

`pop r0`

- 1 Dépiler valeur



Registre R0

Val.

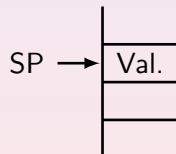
Instructions relatives à la pile

`push r0`

- 1 $SP = SP - 1$
- 2 Empiler valeur

`pop r0`

- 1 Dépiler valeur
- 2 $SP = SP + 1$



Registre R0

Val.

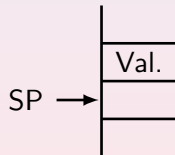
Instructions relatives à la pile

`push r0`

- 1 $SP = SP - 1$
- 2 Empiler valeur

`pop r0`

- 1 Dépiler valeur
- 2 $SP = SP + 1$



Registre R0

Val.

Initialisation de la pile en SimKaal

```
        ORG 0
        jmp debut
        ...
        ORG 20          ; Emplacement de la pile en mémoire
pile:    VAR 10?
fondpile: VAR ?
        ...
debut:   ldr r0,fondpile  ; init. de la pile
        ldr sp,r0
        ...
```

Précautions à prendre

Précautions à prendre

- Dimensionner correctement la taille de la pile, sinon :
 - ▶ Écrasement d'octet(s) en mémoire.

Précautions à prendre

- Dimensionner correctement la taille de la pile, sinon :
 - ▶ Écrasement d'octet(s) en mémoire.
- Ne pas désorganiser la pile, sinon :
 - ▶ Pile qui grossit.
 - ▶ Mauvaise adresse de retour.

Précautions à prendre

- Dimensionner correctement la taille de la pile, sinon :
 - ▶ Écrasement d'octet(s) en mémoire.
- Ne pas désorganiser la pile, sinon :
 - ▶ Pile qui grossit.
 - ▶ Mauvaise adresse de retour.
- Limiter les effets de bords :
 - ▶ Un sous-programme ne doit modifier aucun registre.

Précautions à prendre

- Dimensionner correctement la taille de la pile, sinon :
 - ▶ Écrasement d'octet(s) en mémoire.
- Ne pas désorganiser la pile, sinon :
 - ▶ Pile qui grossit.
 - ▶ Mauvaise adresse de retour.
- Limiter les effets de bords :
 - ▶ Un sous-programme ne doit modifier aucun registre.
 - ▶ Sauvegarde / Restitution avec :
 - ★ push des registres utilisés au début du sous-programme.
 - ★ pop des registres pushés à la fin du sous-programme.

Passage de paramètres

Passage de paramètres

- Par registre :

Passage de paramètres

- Par registre :
 - ▶ Simple à mettre en œuvre.
 - ▶ Registres bloqués.
 - ▶ Nombre de paramètre limité au nombre de registre.

Passage de paramètres

- Par registre :
 - ▶ Simple à mettre en œuvre.
 - ▶ Registres bloqués.
 - ▶ Nombre de paramètre limité au nombre de registre.
- Par mémoire :

Passage de paramètres

- Par registre :
 - ▶ Simple à mettre en œuvre.
 - ▶ Registres bloqués.
 - ▶ Nombre de paramètre limité au nombre de registre.
- Par mémoire :
 - ▶ Nombre illimité de paramètres.
 - ▶ Accès moins rapide que par registre.

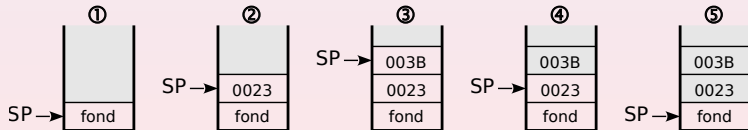
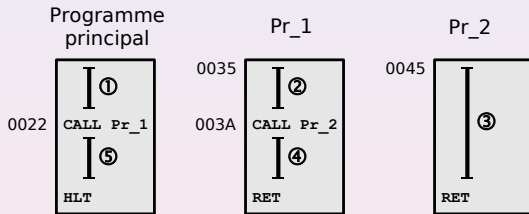
Passage de paramètres

- Par registre :
 - ▶ Simple à mettre en œuvre.
 - ▶ Registres bloqués.
 - ▶ Nombre de paramètre limité au nombre de registre.
- Par mémoire :
 - ▶ Nombre illimité de paramètres.
 - ▶ Accès moins rapide que par registre.
- Par la pile :

Passage de paramètres

- Par registre :
 - ▶ Simple à mettre en œuvre.
 - ▶ Registres bloqués.
 - ▶ Nombre de paramètre limité au nombre de registre.
- Par mémoire :
 - ▶ Nombre illimité de paramètres.
 - ▶ Accès moins rapide que par registre.
- Par la pile :
 - ▶ Nécessite des instructions adaptées.

Exemple d'évolution de la pile



- 1 Prise en main et modes d'adressage
- 2 Sous-programme et pile
- 3 Unité d'échange et transfert par mot d'état**
 - Unité d'échange
 - Les périphériques
 - Transfert par mot d'état
- 4 Les interruptions
- 5 Synthèse

Dialogue avec le monde extérieur

- Communication via périphériques

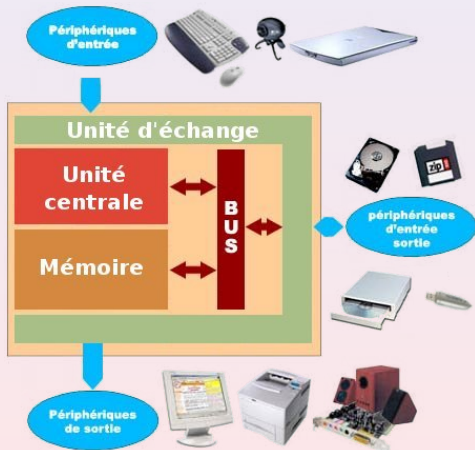
Dialogue avec le monde extérieur

- Communication via périphériques
- Grandes variétés :
 - ▶ vitesse
 - ▶ format

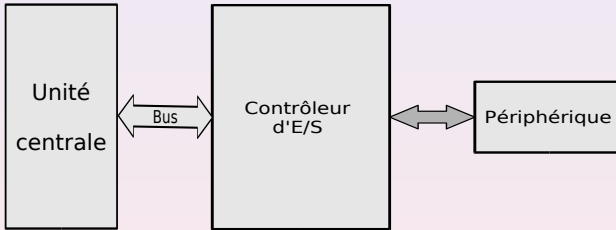
Dialogue avec le monde extérieur

- Communication via périphériques
- Grandes variétés :
 - ▶ vitesse
 - ▶ format

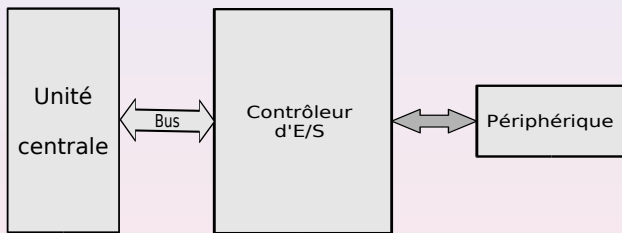
⇒ Circuits spécialisés



Fonctions d'un contrôleur d'entrée-sortie

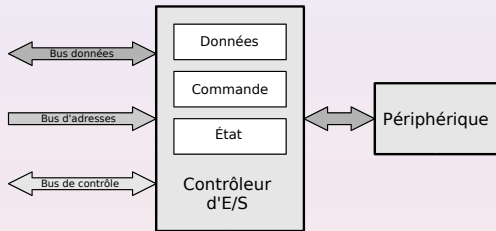


Fonctions d'un contrôleur d'entrée-sortie



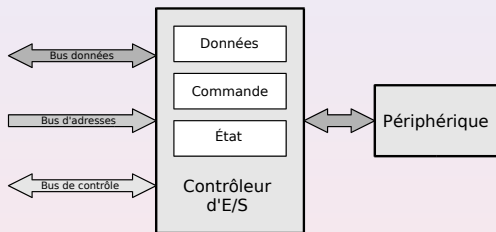
- Permet d'interfacer le CPU avec un périphérique.
- Fournit un moyen standard de dialogue entre le CPU et les périphériques.
- Prend en charge les opérations de bas niveau.
- Libère le CPU de la gestion du périphérique.

Composition d'un contrôleur d'E/S



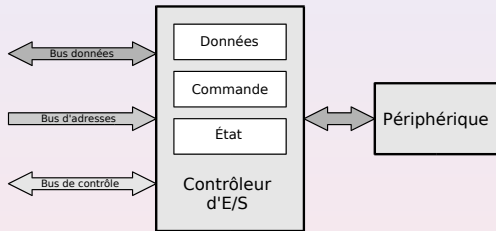
- Registre de données : pour stocker les données échangées.

Composition d'un contrôleur d'E/S



- Registre de données : pour stocker les données échangées.
- Registre de commande :
 - ▶ Programmation du circuit
 - ▶ Accès en écriture lors de la phase d'initialisation

Composition d'un contrôleur d'E/S



- Registre de données : pour stocker les données échangées.
- Registre de commande :
 - ▶ Programmation du circuit
 - ▶ Accès en écriture lors de la phase d'initialisation
- Informations d'état :
 - ▶ Mémorisent la manière dont le transfert s'est effectué
 - ▶ Regroupés dans un registre : le mot d'état (un bit = un état)
 - ▶ Accès en lecture

Espace mémoire des entrées-sorties

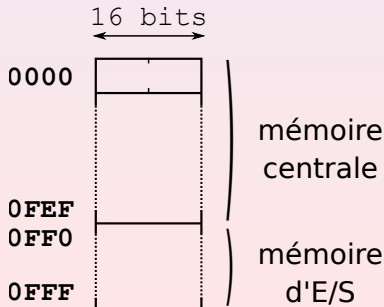
- Soit distinct de la mémoire centrale :
 - ▶ Instruction spéciales (IN, OUT)
 - ▶ Signal de contrôle supplémentaire (IO/M)

Espace mémoire des entrées-sorties

- Soit distinct de la mémoire centrale :
 - ▶ Instruction spéciales (IN, OUT)
 - ▶ Signal de contrôle supplémentaire (IO/M)
- Soit *mappé* en mémoire centrale :
 - ▶ Pas d'instruction ni de signaux spéciaux
 - ▶ SimKaal

Espace mémoire des entrées-sorties

- Soit distinct de la mémoire centrale :
 - ▶ Instruction spéciales (IN, OUT)
 - ▶ Signal de contrôle supplémentaire (IO/M)
- Soit *mappé* en mémoire centrale :
 - ▶ Pas d'instruction ni de signaux spéciaux
 - ▶ SimKaal
- Espace mémoire SimKaal :



Circuit d'E/S parallèle

- Adresses :

FF0	PortA
FF1	PortB
FF2	PortC
FF3	mode

Circuit d'E/S parallèle

- Adresses :
- Port : Registres de données.

FF0	PortA
FF1	PortB
FF2	PortC
FF3	mode

Circuit d'E/S parallèle

- Adresses :

FF0	PortA
FF1	PortB
FF2	PortC
FF3	mode

- Port : Registres de données.
- mode : Pour déterminer le *sens* des ports.

Circuit d'E/S parallèle

- Adresses :

FF0	PortA
FF1	PortB
FF2	PortC
FF3	mode

- Port : Registres de données.
- mode : Pour déterminer le *sens* des ports.
- En entrée :
 - ▶ Connecté à des interrupteurs.
 - ▶ Lecture du registre.
- En sortie :
 - ▶ Connecté à des leds.
 - ▶ Écriture dans registre.

Circuit d'E/S série

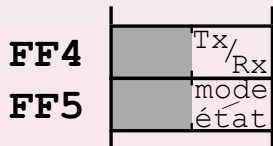
- Adresses :

FF4		Tx/Rx
FF5		mode état

Circuit d'E/S série

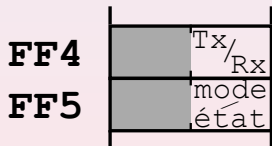
- TX : écriture de 0xFF4
- RX : lecture de 0xFF4

- Adresses :



Circuit d'E/S série

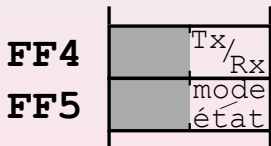
- Adresses :



- TX : écriture de 0xFF4
- RX : lecture de 0xFF4
- mode (écriture de 0xFF5) :
 - ▶ Fonctionnement en IT ou non,
 - ▶ Réception valide ou non,
 - ▶ Transmission valide ou non.

Circuit d'E/S série

- Adresses :



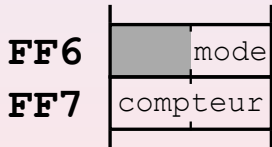
- TX : écriture de 0xFF4
- RX : lecture de 0xFF4
- mode (écriture de 0xFF5) :
 - ▶ Fonctionnement en IT ou non,
 - ▶ Réception valide ou non,
 - ▶ Transmission valide ou non.
- état (lecture de 0xFF5) :



- ▶ RP : Récepteur plein (1) ou vide
- ▶ TV : Transmetteur vide (1) ou occupé
- ▶ ER : Erreur en réception (1) ou non

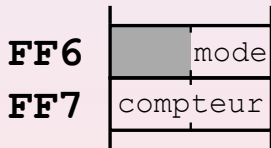
Circuit Temporisateur

- Adresses :



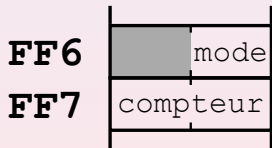
Circuit Temporisateur

- Adresses :
- compteur : Valeur initiale de décomptage (sur 16 bits).



Circuit Temporisateur

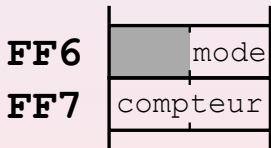
- Adresses :



- compteur : Valeur initiale de décomptage (sur 16 bits).
- mode apériodique :
 - ▶ décompte de la valeur initiale jusqu'à 0,
 - ▶ puis génère une demande d'interruption.

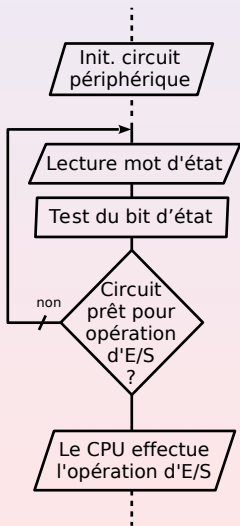
Circuit Temporisateur

- Adresses :

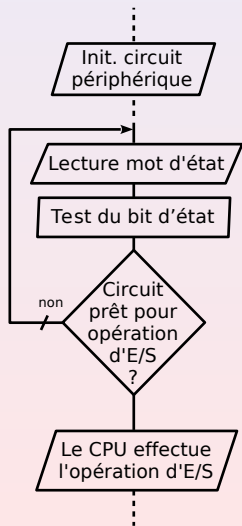


- compteur : Valeur initiale de décomptage (sur 16 bits).
- mode apériodique :
 - ▶ décompte de la valeur initiale jusqu'à 0,
 - ▶ puis génère une demande d'interruption.
- mode périodique :
 - ▶ décompte de la valeur initiale jusqu'à 0,
 - ▶ génère une demande d'interruption,
 - ▶ répète sans fin ce cycle.

Principe



Principe



- Le périph. met à jour ses bits d'états
- Le CPU teste périodiquement l'état qui l'intéresse
- Le périph. n'informe pas le CPU directement
- Le CPU doit attendre ou revenir plus tard

Détail du test du mot d'état (1/2)

- Lecture du mot d'état du périphérique :
 - ▶ Contient plusieurs bits significatifs : b7 b6 b5 b4 b3 b2 b1 b0.
 - ▶ Il faut isoler celui qui nous intéresse : b7 b6 b5 b4 b3 b2 b1 b0.

Détail du test du mot d'état (1/2)

- Lecture du mot d'état du périphérique :
 - ▶ Contient plusieurs bits significatifs : b7 b6 b5 b4 b3 b2 b1 b0.
 - ▶ Il faut isoler celui qui nous intéresse : b7 b6 b5 b4 b3 b2 b1 b0.
- Utilisation d'un masque : 0 0 0 0 0 1 0 0.
- ET logique avec le masque :
 - ▶ Le bit est isolé : 0 0 0 0 0 b2 0 0.
 - ▶ Le résultat vaut 0 si b2 est à 0.
 - ▶ Cette opération positionne l'indicateur Z.

Détail du test du mot d'état (1/2)

- Lecture du mot d'état du périphérique :
 - ▶ Contient plusieurs bits significatifs : b7 b6 b5 b4 b3 b2 b1 b0.
 - ▶ Il faut isoler celui qui nous intéresse : b7 b6 b5 b4 b3 b2 b1 b0.
- Utilisation d'un masque : 0 0 0 0 0 1 0 0.
- ET logique avec le masque :
 - ▶ Le bit est isolé : 0 0 0 0 0 b2 0 0.
 - ▶ Le résultat vaut 0 si b2 est à 0.
 - ▶ Cette opération positionne l'indicateur Z.
- Branchement conditionnel sur l'état de Z :
 - ▶ jz Wait ou jnz Wait
 - ▶ Saut à Wait si le circuit n'est pas prêt.

Détail du test du mot d'état (2/2)

- Exemple :

On souhaite effectuer une opération d'E/S lorsque le bit b2 du mot d'état d'un périphérique passe à 1.

- On souhaite effectuer une opération d'E/S lorsque le bit b2 du mot d'état d'un périphérique passe à 1.

- PS Etat EQU 0xFF5 ; Équivalence sur le mot d'état

```
Wait:  ldr r1,[PS_Etat]    ; lecture mot d'état
```

`jz Wait` ; si `b2` est à 0, on attends

Pret: ... ; b2 est à 1, on effectue l'opération

- 1 Prise en main et modes d'adressage
- 2 Sous-programme et pile
- 3 Unité d'échange et transfert par mot d'état
- 4 Les interruptions**
 - Introduction
 - Le contrôleur d'interruption
 - Déroulement d'une interruption
 - Les interruptions de SimKaal
- 5 Synthèse

Concept d'interruption

Jusqu'à présent :

- Exécution séquentielle
- Branchements programmés
- E/S programmées

Concept d'interruption

Jusqu'à présent :

- Exécution séquentielle
- Branchements programmés
- E/S programmées

On souhaite un comportement plus dynamique face à :

- Des évènements internes
- Des évènements matériels externes
- Des instructions spécifiques

Dispositions pour un fonctionnement par it

Dispositions pour un fonctionnement par it

- Signaux électriques (pour les it externes)
 - ▶ NMI : it non masquables
 - ▶ INTR : it masquables

Dispositions pour un fonctionnement par it

- Signaux électriques (pour les it externes)
 - ▶ NMI : it non masquables
 - ▶ INTR : it masquables
- Registre ou Indicateur

Dispositions pour un fonctionnement par it

- Signaux électriques (pour les it externes)
 - ▶ NMI : it non masquables
 - ▶ INTR : it masquables
- Registre ou Indicateur
- Jeu d'instructions
 - ▶ `ldr fl,r0` : pour manipuler le flag (pour les it masquables)
FL

	Z	C	I
--	---	---	---
 - ▶ `int n` : déclenchement d'une it logicielle.

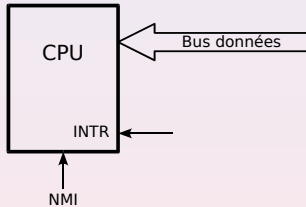
Comparatif pour les E/S

- Transfert par mot d'états :
 - ▶ le CPU a un contrôle direct sur le périphérique
 - ▶ Il doit attendre que le périphérique ait fini son opération
 - ▶ $\Rightarrow$ Grosse perte de temps CPU
 - ▶ Simple à mettre en œuvre

Comparatif pour les E/S

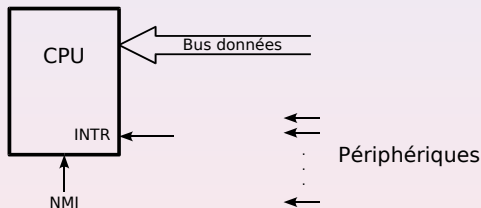
- Transfert par mot d'états :
 - ▶ le CPU a un contrôle direct sur le périphérique
 - ▶ Il doit attendre que le périphérique ait fini son opération
 - ▶ $\Rightarrow$ Grosse perte de temps CPU
 - ▶ Simple à mettre en œuvre
- Transfert par interruption :
 - ▶ Nécessite plus de matériel
 - ▶ Pas d'attente

Prise en compte de plusieurs interruptions



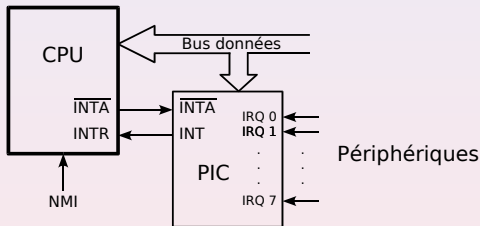
- Le CPU : une seule broche d'interruption

Prise en compte de plusieurs interruptions



- Le CPU : une seule broche d'interruption
- Le système : plusieurs périphériques

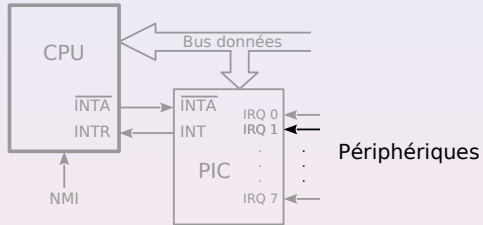
Prise en compte de plusieurs interruptions



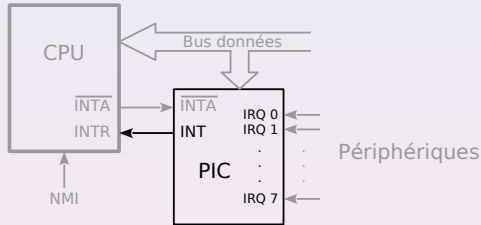
- Le CPU : une seule broche d'interruption
- Le système : plusieurs périphériques
- $\Rightarrow$ Utilisation d'un circuit spécialisé (PIC)

Priorité et niveau

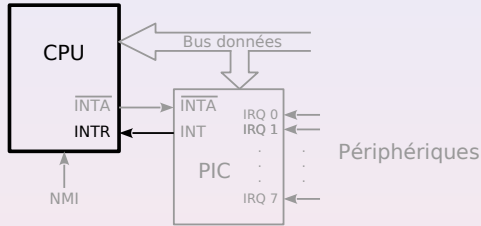
- Circuit capable de gérer plusieurs entrées de demande d'it.
- Ne fait suivre qu'une seule it vers le CPU.
 - ▶ Notion de priorité entre it
- Chaque entrée est associée à un niveau.
 - ▶ Permet de gérer la priorité.
- Le niveau d'interruption :
 - ▶ Est transmis au CPU,
 - ▶ Le CPU l'utilise pour trouver l'adresse du sous-programme d'it.



Une interruption est émise par un périphérique sur une entrée du PIC.

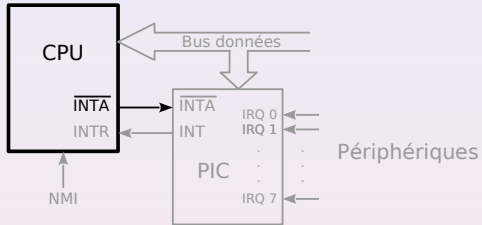


Si l'entrée est démasquée, le contrôleur propage la demande et génère un signal sur sa broche INT.

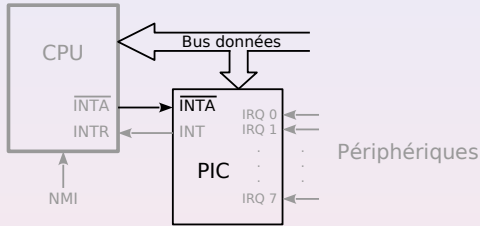


Le CPU termine l'instruction en cours d'exécution.

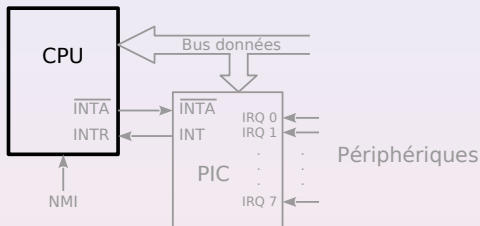
Si l'indicateur I vaut 0, le signal sur la borne $INTR$ est ignoré, sinon, la demande d'interruption est acceptée.



Si la demande est acceptée, le CPU active sa sortie INTA .



En réponse, le contrôleur d'interruption place le niveau de l'interruption associé à la borne IRQ sur le bus de donnée.



Pour exécuter le sous-programme associé à l'interruption, le CPU :

- Sauvegarde des indicateurs (registre FL : Z, C, I)
- Positionne I à 0
- Sauvegarde le registre IP en pile
- Se branche à l'adresse du sous-programme d'interruption
- Sous-programme d'interruption :
 - ▶ Gestion de l'EOI (*End Of Interrupt*)
 - ▶ `iret` $\Rightarrow$ restitution de l'adresse de retour et des indicateurs.

La table des vecteurs d'interruption.

- Zone mémoire située à l'adresse 0.
- Les éléments de ce tableau sont les adresses des sous-programmes d'interruption.
- Le niveau d'interruption sert d'index dans la table des vecteurs.

La table des vecteurs d'interruption.

- Zone mémoire située à l'adresse 0.
- Les éléments de ce tableau sont les adresses des sous-programmes d'interruption.
- Le niveau d'interruption sert d'index dans la table des vecteurs.

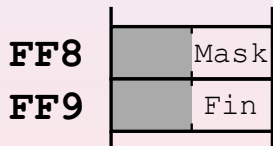
Exemple avec des adresses sur 2 octets :

Niveaux	Adresses	Mémoire
0	0000	@ sp it 0
1	0001	@ sp it 1
2	0002	@ sp it 2

Figure – Table des vecteurs d'interruptions

Contrôleur d'interruption

- Adresses :

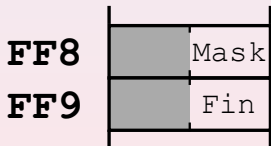


Contrôleur d'interruption

- Mask :



- Adresses :



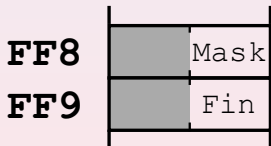
- ▶ $Im\langle n \rangle$ à 0 :
entrée démasquée $\Rightarrow$ demande propagée
- ▶ $Im\langle n \rangle$ à 1 :
entrée masquée $\Rightarrow$ demande bloquée

Contrôleur d'interruption

- Mask :



- Adresses :



- ▶ $\text{Im}\langle n \rangle$ à 0 :
entrée démasquée $\Rightarrow$ demande propagée
- ▶ $\text{Im}\langle n \rangle$ à 1 :
entrée masquée $\Rightarrow$ demande bloquée

- Fin :

- ▶ Permet de gérer l'imbrication (éventuelle) de plusieurs interruptions.
- ▶ Valeur 0 : l'interruption en cours est terminée,
- ▶ Les demandes en attentes sont alors traitées.

- 3 sources d'interruptions :
 - ▶ IR1 : Activé par la réception d'une donnée sur le circuit d'E/S série.
 - ▶ IR2 : Activé par bouton poussoir à cliquer.
 - ▶ IR3 : Activé par le temporisateur (passage à 0).

- 3 sources d'interruptions :
 - ▶ IR1 : Activé par la réception d'une donnée sur le circuit d'E/S série.
 - ▶ IR2 : Activé par bouton poussoir à cliquer.
 - ▶ IR3 : Activé par le temporisateur (passage à 0).
- Gestion de la priorité :
 - ▶ IR3 est l'entrée la plus prioritaire.
 - ▶ En cas de demande simultanée, la plus prioritaire est prise en compte.

- 3 sources d'interruptions :
 - ▶ IR1 : Activé par la réception d'une donnée sur le circuit d'E/S série.
 - ▶ IR2 : Activé par bouton poussoir à cliquer.
 - ▶ IR3 : Activé par le temporisateur (passage à 0).
- Gestion de la priorité :
 - ▶ IR3 est l'entrée la plus prioritaire.
 - ▶ En cas de demande simultanée, la plus prioritaire est prise en compte.
- Table des vecteurs :

IR	Niv.		Mémoire	Code
	0	0000	A0 03	
1	1	0001	@ sp it 1	ORG 0 jmp debut
2	2	0002	@ sp it 2	
3	3	0003	@ sp it 3	ORG 4
		0004		debut: ...

Exemple de programme fonctionnant en it

```
EOI      EQU 0xFF9
        ORG 0
        jmp init
; Table des vecteurs
vect1: VAR ?           ; vecteur IT E/S série
vect2: VAR ?           ; vecteur IT Bouton poussoir
vect3: VAR ?           ; vecteur IT Temporisateur
init:    ...           ; inhib IT CPU (FL <- 0)
        ldr r0,it_bp   ; init TdV
        str [vect2],r0
        ...           ; init pile
        ...           ; init periph en it
        ...           ; init mask du PIC
        ...           ; valide IT CPU (FL <- 1)
debut:   ...
it_bp:   push ...      ; sauvegarde registres utilisés
        ...
        ldr r0,0        ; End Of Interrupt
        str [EOI],r0
        pop ...         ; restitution registres
        iret
```

Imbrication des interruptions

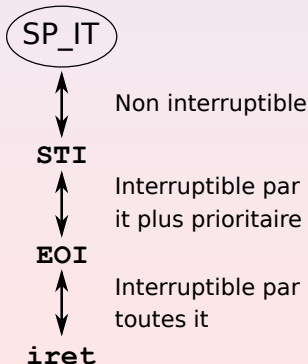
- Par défaut, un SP_IT est non-interruptible.

Imbrication des interruptions

- Par défaut, un SP_IT est non-interruptible.
- Pour le rendre interruptible :
 - ▶ Valider les it au niveau du CPU ($\approx$ STI)
 - ▶ Gérer l'EOI

Imbrication des interruptions

- Par défaut, un SP_IT est non-interruptible.
- Pour le rendre interruptible :
 - ▶ Valider les it au niveau du CPU ($\approx$ STI)
 - ▶ Gérer l'EOI
- 3 zones :



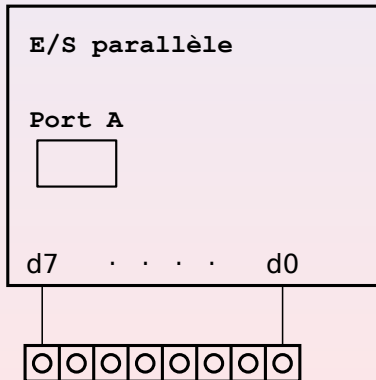
- 1 Prise en main et modes d'adressage
- 2 Sous-programme et pile
- 3 Unité d'échange et transfert par mot d'état
- 4 Les interruptions
- 5 **Synthèse**
 - Afficheur 7 segments
 - Gestion d'un parking

Principe

- Périphérique du circuit d'E/S //

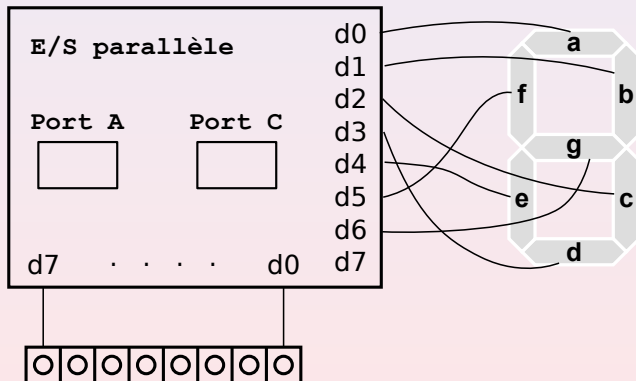
Principe

- Périphérique du circuit d'E/S //
- Port A sur leds :



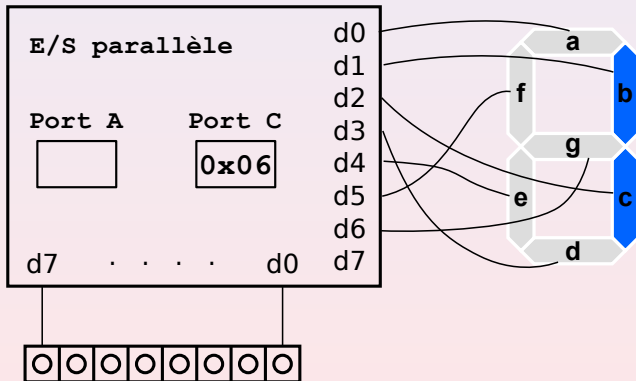
Principe

- Périphérique du circuit d'E/S //
- Port C sur 7 segments :



Principe

- Périphérique du circuit d'E/S //
- Port C sur 7 segments : affichage de 1



Utilisation

- Tableau de codage

	h	g	f	e	d	c	b	a	Port C
0	0	0	1	1	1	1	1	1	0x3F
1	0	0	0	0	0	1	1	0	0x06
2	0	1	0	1	1	0	1	1	0x5B
3	0	1	0	0	1	1	1	1	0x4F
4	0	1	1	0	0	1	1	0	0x66

Utilisation

- Exemple de code pour afficher un 2 :

```
PortC EQU 0xFF2
```

```
...
```

```
ORG 10 ; Codage pour affichage 7 segments
```

```
tseg: CST 0x3F,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x7,0x7f,0x6f
```

```
...
```

```
ldr r0,tseg ; Adresse de base du tableau
```

```
ldr r1,r0
```

```
ldr r0,2 ; Offset dans le tableau
```

```
add r4,r1r0 ; Calcul de l'adresse du '2'
```

```
ldr r2,[r4] ; Code du '2' : 0x5b
```

```
str [PortC],r2 ; Affichage sur 7 segments
```

Illustration

